CSTE4103 — Cloud Computing

Distributed Systems and Cloud-Native Software

Consensus and Raft, Consistency and the CAP Theorem, Cloud-Native vs Conventional Software, MapReduce, Hadoop, and HDFS

Lecture 5 • Cybersecurity Engineering — 4th Year

Dr. Ruaa Ali Khamees • ICT Department
Polytechnic College for Engineering Specializations – Baghdad, Middle Technical University

LECTURE 5 • OVERVIEW

Learning Objectives

1. **Why agreement is hard**
   Explain failures in distributed systems and why nodes must reach consensus.
2. **Raft and Paxos**
   Describe quorums, Raft leader election, and log replication.
3. **Consistency and CAP**
   Compare consistency levels and apply the CAP theorem to real systems.
4. **Cloud-native software**
   Contrast cloud-native and conventional software.
5. **MapReduce, Hadoop, HDFS**
   Explain the MapReduce model with its math, and the HDFS architecture.
6. **Security**
   Evaluate security risks of consensus clusters, stale reads, and big data platforms.

1 — WHY

Why Distribute?

One computer cannot hold all the data, serve all the users, or survive its own failure. Clouds spread work across many machines, and that creates new problems.

Scale

Data and traffic exceed what one server can store or process. Add machines instead of buying a bigger one.

Availability

If one machine dies, another continues. Replicas keep the service running.

Locality

Place data near users and near the computation to cut latency.

The price: machines fail independently, messages are delayed or lost, and clocks disagree. So how do many machines agree on one answer? That is the topic of today.

1 — WHY

What Can Go Wrong

Failure	Meaning	Example
Crash	A node stops and may later restart	Power loss, kernel panic
Message loss or delay	A message arrives late, twice, or never	Congested link, packet drop
Network partition	The cluster splits into groups that cannot talk	A switch or inter-site link fails
Clock skew	Nodes disagree about time, so event order is unclear	Drifting server clocks
Byzantine	A node lies or behaves arbitrarily	Compromised or buggy node

⚠ Security angle: Raft and Paxos tolerate crashes and lost messages, but not Byzantine nodes. A compromised member can break the guarantees, so membership must be authenticated.

The Need for Agreement

Consensus means a group of nodes agrees on a single value, even when some nodes fail. Many cloud services depend on it.

Where it is needed

- Electing one leader (who is in charge?)
- A replicated log of operations in the same order
- Distributed locks and configuration
- Committing a transaction across nodes

Required properties

- Agreement: no two nodes decide differently
- Validity: the decided value was proposed by someone
- Termination: every working node eventually decides

Example: three replicas of a grades database. A client writes grade 85, and another writes 90 at the same moment. All replicas must apply the writes in the same order, or they will end with different data.
A known result (FLP) says no algorithm can guarantee agreement in a fully asynchronous network with even one crash. Real protocols use timeouts to work around it.

Quorum: The Power of a Majority

A cluster of 5 nodes: 3 form a majority



Majority (3 of 5): can decide **Minority (2 of 5): cannot decide**

Nodes (N)	Majority	Failures tolerated
3	2	1
4	3	1
5	3	2
7	4	3

Why it works

- Any two majorities share at least one node, so two groups can never decide differently
- With N = 2f + 1 nodes, the cluster survives f crashes
- A partition leaves at most one side with a majority

Use an odd number of nodes. 4 nodes tolerate only 1 failure, the same as 3, and cost more. Spread nodes across failure domains (racks, sites).

2 — CONSENSUS

Paxos in Brief

Paxos (Lamport) is the classic consensus algorithm. It is proven correct but famously hard to understand and implement.

Proposer

Suggests a value and drives the protocol.

Acceptor

Votes. A value is chosen when a majority of acceptors accept it.

Learner

Finds out which value was chosen and applies it.

Phase 1: Prepare

Proposer sends Prepare(n) with a number n. Acceptors reply Promise(n): they will ignore lower numbers and report any value already accepted.

Phase 2: Accept

With a majority of promises, the proposer sends Accept(n, v). Acceptors accept unless they promised a higher number. A majority of accepts means v is chosen.

In practice: Multi-Paxos runs many rounds with a stable leader. Used inside Google Chubby and Spanner, and in Ceph monitors. Raft was designed to be easier to understand.

2 — CONSENSUS

Raft: Consensus Made Understandable

Raft keeps one strong leader. Clients talk to the leader, and the leader replicates every change to the followers.



Each node is in exactly one state at a time. A split vote means a new election.

Raft splits the problem into three parts: leader election, log replication, and safety.

Term

A logical clock. Each election starts a new term.

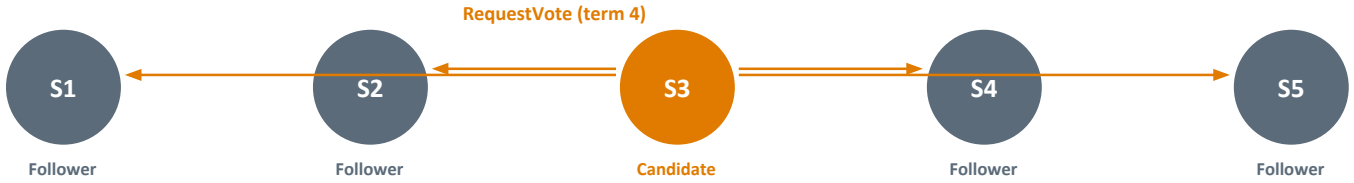
Two RPCs

RequestVote and AppendEntries.

Heartbeat

An empty AppendEntries the leader sends regularly to stay in charge.

Raft: Leader Election



1 Timeout

The leader goes silent. Followers wait a random timeout (about 150 to 300 ms).

2 Candidate

The first to time out increments the term, votes for itself, and asks others for votes.

3 Vote

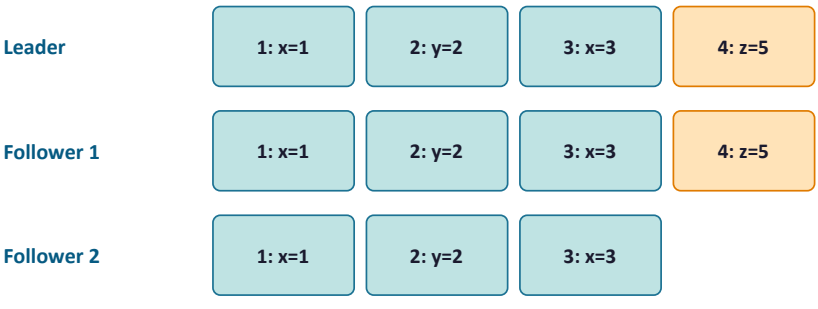
Each node votes at most once per term, for the first valid request.

4 Leader

With 3 of 5 votes, S3 becomes leader and sends heartbeats immediately.

Split vote: if two candidates tie, nobody wins. Random timeouts make a second tie unlikely, so a new election resolves it quickly.

Raft: Log Replication



Entry 4 is on the Leader and Follower 1 = 2 of 3 nodes = majority, so it is committed.

Why this is safe

- Only the leader appends entries, so the order is decided in one place
- An entry is committed only after a majority stores it
- Followers reject entries that do not match their previous entry, so logs stay consistent
- A lagging follower is caught up automatically

1 Client

sends a command to the leader

2 Append

leader adds it to its own log

3 Replicate

AppendEntries to followers

4 Commit

majority stored it: commit

5 Reply

apply to state machine, answer client

Raft Under Failure

Scenario	What happens	Why it is safe
Leader crashes	Followers stop receiving heartbeats, time out, and elect a new leader in a higher term	Committed entries are on a majority, and the new leader must have them
Follower crashes	The leader keeps working while a majority is alive. The follower catches up when it returns	No decision needs every node
Partition 3 | 2	The side with 3 nodes keeps a leader and accepts writes. The side with 2 cannot commit anything	Only one side has a majority, so there is no split brain
Old leader returns	It sees a higher term and steps down to follower	Terms order leaders. Stale leaders cannot commit

Election restriction: a candidate whose log is behind cannot win. This protects committed data.

Where Consensus Lives in the Cloud

System	Protocol	What it keeps in agreement
etcd	Raft	All Kubernetes cluster state, including Secrets
Consul	Raft	Service discovery and configuration
ZooKeeper	ZAB (Paxos-like)	Coordination for Hadoop, Kafka, HBase: locks, leaders
Ceph	Paxos	The cluster map held by the monitors
Spanner, CockroachDB	Paxos, Raft	Replicated data ranges in a global SQL database

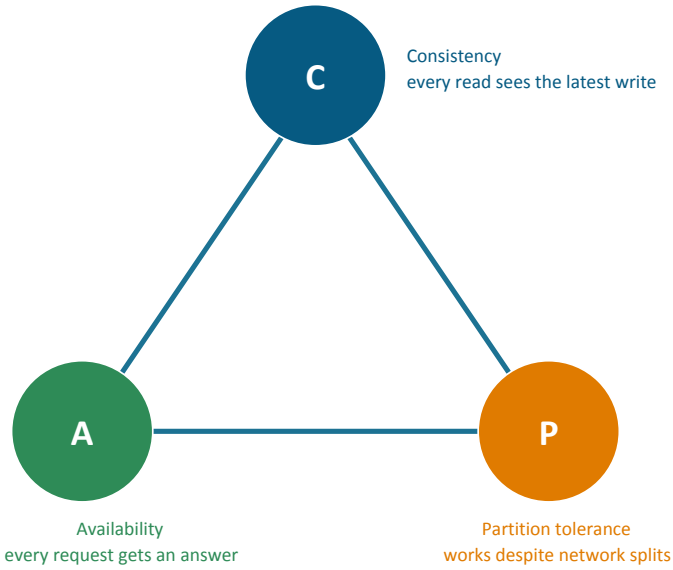
Link to Lecture 4: SDN controller clusters and storage metadata also rely on consensus. Rule of thumb: use a proven system such as etcd, do not write your own consensus.

Consistency Levels

Level	Guarantee	Example	Cost
Strong (linearizable)	Every read returns the latest write, as if there were one copy	Bank balance, leader lock	Highest latency; may refuse requests during a partition
Sequential	Everyone sees operations in the same order, not necessarily in real time	Replicated log	Slightly cheaper than strong
Causal	If A caused B, everyone sees A before B	A comment appears after its post	Moderate
Eventual	Replicas converge if writes stop, but reads may be stale meanwhile	DNS, like counters, shopping cart	Fastest and most available

⚠ Security angle: With eventual consistency, a revoked permission or disabled account can still be honored by a lagging replica. Use strong reads for authorization.

The CAP Theorem



What it really says

Partitions will happen in any real network. When one does, you must choose: stay consistent (reject some requests) or stay available (answer, maybe with stale data).

Choice	Behavior during a partition	Examples
CP	Minority side refuses requests	etcd, ZooKeeper, HBase
AP	Every side answers, replicas reconcile later	Cassandra, DynamoDB, CouchDB

Without a partition, the trade-off is latency versus consistency (PACELC).

4 — CLOUD-NATIVE SOFTWARE

Cloud-Native vs Conventional Software

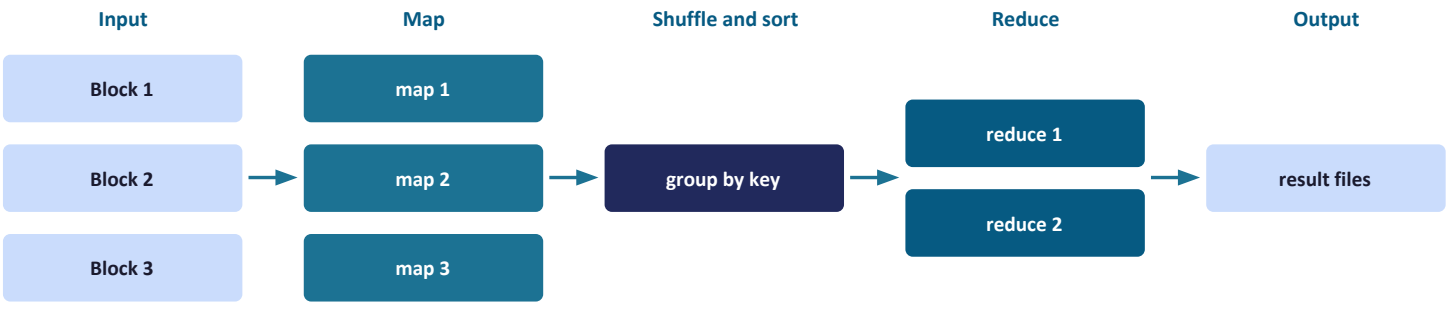
Aspect	Conventional	Cloud-native
Architecture	One large monolithic application	Small independent microservices
Runtime	Installed on a specific server or VM	Containers managed by an orchestrator such as Kubernetes
Scaling	Scale up: a bigger server	Scale out: more small instances, often automatic
Failure	Avoided; assumes reliable hardware	Expected; designed to self-heal
State	Stored on the local server	Services are stateless; state lives in external stores
Releases	Rare, large, manual	Frequent, small, automated (CI/CD)

Cloud-native does not mean running in the cloud. It means designed to exploit the cloud: elastic, resilient, automated.

5 — PARALLEL PROCESSING

MapReduce: Divide, Process, Combine

When data is too large for one machine, split it, process the pieces in parallel on many machines, then combine the results. MapReduce is a programming model that does this for you.



Bring computation to the data

Map tasks run on the machines that already store the block, so little data crosses the network.

Failures are handled

If a task fails or is slow, the framework simply runs it again on another machine.

5 — PARALLEL PROCESSING

Word Count and the Math of MapReduce

Step	Data
Input lines	L1: cloud is big L2: cloud is shared L3: big data is big
Map emits (word, 1)	(cloud,1) (is,1) (big,1) (cloud,1) (is,1) (shared,1) (big,1) (data,1) (is,1) (big,1)
Shuffle groups by key	big:[1,1,1] cloud:[1,1] data:[1] is:[1,1,1] shared:[1]
Reduce sums	big 3 cloud 2 data 1 is 3 shared 1

Job time ≈ T_map / N + T_shuffle + T_reduce / R
Adding machines shrinks the map and reduce parts, but the shuffle (network) and the serial part do not shrink. That limits speedup.

The model

map: (k1, v1) → list(k2, v2)

reduce: (k2, list(v2)) → list(v3)

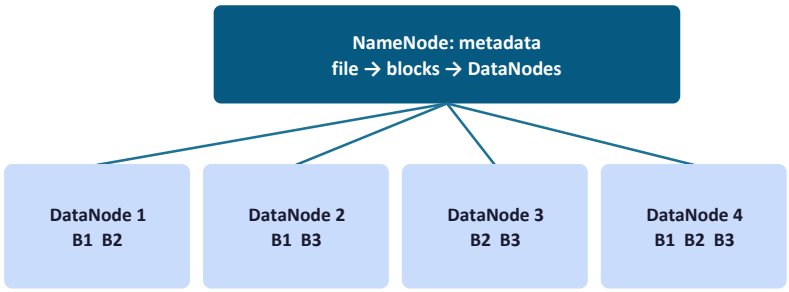
Speedup (Amdahl)

S(N) = 1 / ((1 − p) + p / N)
p = parallel fraction, N = machines
Example: p = 0.95, N = 100 gives S ≈ 16.8, not 100

Check: big 3 = 1 (L1) + 2 (L3). Total words 10 = 3 + 2 + 1 + 3 + 1.

5 — PARALLEL PROCESSING

Hadoop and HDFS



A file is cut into 128 MB blocks. Each block is stored on 3 different DataNodes (replication, as in Lecture 4). Clients read data directly from DataNodes.

Hadoop = three layers

- HDFS: distributed storage
- YARN: resource scheduling for the cluster
- MapReduce: the processing engine

Languages

Java natively. Hadoop Streaming lets any language that reads stdin and writes stdout (Python, Perl) act as mapper or reducer. Hive and Pig add SQL-like and scripting layers.

The NameNode is a single point of failure. High availability uses an active and a standby NameNode, with edits kept in a quorum of JournalNodes and failover coordinated by ZooKeeper. That is consensus again.

6 — SECURITY LENS

Security Lens: Distributed Systems

Area	Typical risk	Good practice
Consensus cluster (etcd, ZooKeeper)	Unauthenticated members or clients; whoever controls a majority controls the state, including Kubernetes Secrets	Mutual TLS, client authentication, encryption at rest, network isolation
Consistency and CAP	Stale reads honor revoked permissions; split brain after a bad failover	Strong reads for authorization, short token lifetimes, fencing tokens
Hadoop and HDFS	Open web UIs and RPC ports, default weak authentication, data exposed to every job	Kerberos, ACLs, encryption zones, audit logs, isolated network
Cloud-native apps	Vulnerable images, secrets in code, over-privileged service accounts, supply-chain attacks	Image scanning and signing, secret managers, least-privilege accounts
MapReduce jobs	Untrusted code runs on the cluster; poisoned input data skews results	Tenant isolation, quotas, input validation, sandboxing

7 — APPLY IT

Mini Case: A Results Service Across Three Sites

Scenario: a university runs an exam-results service replicated across three sites. It stores grades and a "last login" time for each student. Each night it must analyze 500 GB of access logs to find students with many failed logins.

Q1 CP or AP

Which consistency would you choose for grades, and which for last login? Why?

Q2 Raft

How many Raft nodes, and where? What happens if one site is cut off?

Q3 MapReduce

Describe the map and reduce functions to count failed logins per student.

Q4 Threats

Name two threats to this design and one control for each.

SUMMARY

Key Takeaways

- Consensus lets failing nodes agree. A majority quorum makes decisions safe: N = 2f + 1 survives f failures
- Raft elects one leader, replicates a log, and commits once a majority stores each entry
- CAP: during a partition choose consistency or availability. Pick the level per data type
- Cloud-native software is built from small stateless services that expect failure and scale out
- MapReduce splits work into map and reduce; HDFS stores replicated blocks; secure both

Next lecture

Microservices: monoliths in data centers, the microservices approach (pros and cons), granularity, communication protocols, service mesh proxy, and technologies.

Reference: Comer, D. E. (2021). The Cloud Computing Book. CRC Press.